

CoEvoP&R: Co-Evolving Placement Objectives with Routing Feedback via Large Language Models

Anonymous Author(s)

Abstract

Analytical placers optimize half-perimeter wirelength (HPWL) and density, but recent post-route benchmarking studies show that these proxies correlate poorly with routed wirelength, routing congestion, and timing. Existing routing-aware placers handle this gap with fixed expert terms or learned black-box surrogates, leaving the differentiable objective itself fixed. CoEvoP&R instead makes complete replacement objectives the search target. A large language model (LLM) proposes readable symbolic programs, DREAMPlace measures their optimizer behavior, and scheduled routed promotions condition later proposals. This optimizer-embedded loop requires every candidate to pass a restricted objective interface, autograd checks, term-compatibility rules, and a cost-scaled placement/routing cascade. A perturbation-based proxy audit gives cheap timing proxies selection influence only when they are non-redundant with HPWL and agree with downstream timing. CoEvoP&R is evaluated on ChiPBench Nangate45, ICCAD 2015 Superblue, and ASAP7 cross-node transfer. On the primary ChiPBench benchmark, CoEvoP&R reduces DREAMPlace HPWL by 16.9% and density overflow by 36.7% on average over native DREAMPlace, while improving OpenTimer placement-RC WNS by 0.70 ns and TNS by 912 ns. Each search returns a compact symbolic objective that plugs directly into DREAMPlace.

Keywords

electronic design automation, placement, routing, large language models, objective functions

ACM Reference Format:

Anonymous Author(s). 2027. CoEvoP&R: Co-Evolving Placement Objectives with Routing Feedback via Large Language Models. In *Proceedings of the 32nd Asia and South Pacific Design Automation Conference (ASP-DAC 2027)*, January 25–28, 2027, Tokyo, Japan. ACM, New York, NY, USA, 8 pages.

1 Introduction

Modern analytical placers make global placement tractable by optimizing half-perimeter wirelength (HPWL) plus a density penalty. These cheap, smooth proxies enable gradient-based tools such as DREAMPlace [14]. However, the same objective choice also limits what the placer can optimize. ChiPBench shows that placement-stage HPWL correlates poorly with post-route quality on modern industrial designs, including routed wirelength, routing congestion, worst negative slack (WNS), total negative slack (TNS), and design-rule checking (DRC) count [26]. The analytical placer optimizes one target while the completed flow depends on another.

Routing-aware placement has addressed this mismatch through two main changes to the objective. The first adds hand-engineered symbolic terms on top of a fixed placement objective. Rectangular

Uniform wire DensitY (RUDY) estimates routing demand from placement geometry [24], and later congestion and density controls build on the same idea. Terms of this kind sharpen routing awareness but fix the objective form and require expert redesign for each technology or benchmark family. The second inserts learned routing or timing surrogates into placement. LaMPlace improves post-route WNS by 43.0 percent and TNS by 30.4 percent [9]. GOALPlace reduces DRC violations by up to tenfold [1]. RoutePlacer lowers routing congestion while preserving routed wirelength [11]. These methods add downstream awareness but embed part of the objective in black-box models that engineers cannot inspect or edit. Neither change makes the differentiable placement objective itself searchable. This leaves an open third path: direct search over readable, differentiable objective forms that still run inside analytical placement.

CoEvoP&R searches this third path with LLM program evolution. FunSearch, Eureka, LLM-SR, and AlphaEvolve show that LLMs can revise executable programs using automatic evaluator feedback [18, 20, 21, 23]. EvoPlace and VeoPlace bring foundation-model search into placement by evolving optimizer components or guiding macro-placement actions [25, 27]. These systems search initialization, optimizer, or action choices around the analytical objective. CoEvoP&R searches the objective itself: an LLM proposes complete symbolic replacement objectives, and DREAMPlace evaluates each safe candidate inside the gradient path that produces a placement. This embedding makes optimizer compatibility part of the search space. Static admission, diagnostic autograd checks, and runtime guards reject incompatible or unstable objectives. Because full OpenROAD routing is too expensive for every candidate, scheduled promotions return routed evidence to later proposals [3]. A proxy audit gives cheap timing signals selection influence only after they are non-redundant with HPWL and agree with downstream timing.

CoEvoP&R makes four contributions.

- **Searchable objectives.** CoEvoP&R searches complete differentiable replacement objectives and discovers readable formulas that improve placement and post-GRT quality over the native DREAMPlace objective.
- **Optimizer-embedded evolution.** CoEvoP&R inserts each LLM objective into the DREAMPlace gradient path through static, diagnostic, and runtime safety checks.
- **Proxy audit.** CoEvoP&R gives cheap timing proxies selection influence only after non-redundancy and routed-agreement checks.
- **Deployable artifact.** Each CoEvoP&R search returns a compact symbolic objective that engineers can inspect, edit, and plug into DREAMPlace.

On the primary Nangate45 panel, CoEvoP&R reduces DREAMPlace HPWL by 16.9% and density overflow by 36.7% on average,

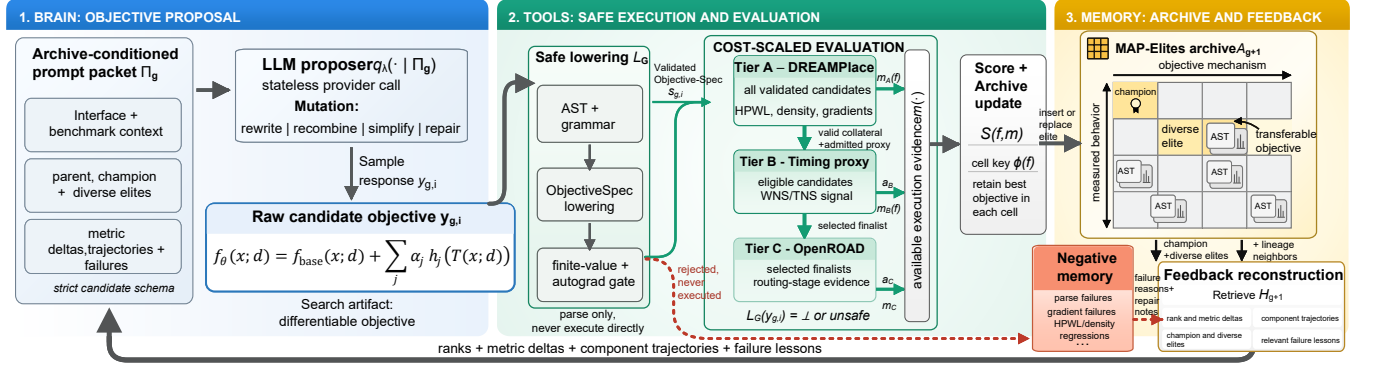


Figure 1: CoEvoP&R evolves complete placement objectives through archive-conditioned proposal, safe EDA evaluation, and scheduled routed feedback.

while improving OpenTimer placement-RC WNS by 0.70 ns and TNS by 912 ns.

2 Related Work

Routability-Aware Objectives. Routability-aware placement exposes downstream routing or timing cost to an analytical optimizer. RePlAce established the electrostatic density formulation [6], and DREAMPlace made analytical placement GPU-efficient in PyTorch [14]. One line adds expert symbolic terms: RUDY estimates routing demand from placement geometry [24]. A second line inserts learned downstream surrogates. LaMPlace, GOALPlace, and RoutePlacer connect placement to post-route timing, design-rule checking (DRC), or congestion through masks, graph neural networks, and learned routability signals [1, 9, 11]. These methods add downstream awareness through fixed expert terms or learned components. CoEvoP&R searches for readable symbolic objective forms that still execute inside the analytical placer.

Foundation-Model Placement Search. Placement search methods differ by the artifact they modify. Reinforcement learning places macros sequentially [19]. AutoDMP tunes macro-placement parameters with multi-objective Bayesian optimization [2], and HyperPlace uses an off-the-shelf LLM to tune GPU-placer hyperparameters [5]. EvoPlace evolves optimizer code, including initialization, preconditioning, and update components, with HPWL feedback [27]. VeoPlace uses a vision-language model (VLM) to guide macro-placement actions through evolutionary search [25]. ChiP-Bench provides end-to-end validation for whether placement gains survive downstream flow [26]. These systems search parameters, optimizer components, or macro actions around the placement objective. CoEvoP&R targets the differentiable objective itself and uses scheduled routed promotions to condition later objective proposals.

Executable LLMs for EDA. LLM-based EDA work often uses models for text, register-transfer level (RTL), or prediction tasks. ChipNeMo adapts foundation models for chip-design tasks [16]. RTLcoder generates Verilog from natural language [17]. MasterRTL and VeriLoC predict pre-synthesis power, performance, area, or line-level design quality from Verilog [8, 10]. A second line turns LLM output into executable programs. FunSearch searches over Python

programs for combinatorial mathematics [21]. Eureka evolves reward functions from training-statistics feedback [18]. Evolution of Heuristics (EoH) co-evolves natural-language thoughts and executable heuristic code [15]. LLM-SR recovers symbolic scientific equations under automated evaluation [23]. AlphaEvolve extends the pattern to full code files across mathematics and computing infrastructure [20], and OpenEvolve provides a public implementation [22]. These systems evaluate generated programs after execution. CoEvoP&R places generated code inside DREAMPlace’s autograd path, so validity requires finite objective values, finite gradients, and optimizer-compatible terms before routed validation.

3 Method

CoEvoP&R searches the differentiable placement objective while keeping DREAMPlace’s optimizer fixed. This isolates objective quality and permits new interactions among wirelength, density, routing pressure, and timing-relevant observables that fixed-form parameter tuning cannot express. Figure 1 shows the loop: archive-conditioned proposal, restricted lowering, cost-scaled EDA evaluation, and feedback reconstruction. The LLM neither invokes the EDA tools nor selects fitness.

3.1 Problem Formulation

Let $d \in \mathcal{D}$ denote a design, s a placement seed, and $\mathbf{x}$ the positions of all movable macros and standard cells. A candidate objective f produces the mixed-size placement

$$\mathbf{x}_{d,s}(f) = \mathcal{P}_{d,s}(f), \quad (1)$$

where $\mathcal{P}_{d,s}$ is DREAMPlace with its native objective replaced by f ; the optimizer and downstream evaluators remain fixed.

Let $\mathcal{T} = \{t_1, \dots, t_k\}$ contain the allowed differentiable observables. The first call records each term scale κ_j , which remains fixed for the run. A complete replacement objective is

$$\tilde{t}_j(\mathbf{x}; d) = \frac{t_j(\mathbf{x}; d)}{\kappa_j}, \quad f_\theta(\mathbf{x}; d) = \kappa_{wl} F_\theta(\tilde{\mathcal{T}}(\mathbf{x}; d)), \quad (2)$$

where restricted program F_θ has one wirelength and one density anchor; routing terms and nonlinear interactions are optional. κ_{wl} restores wirelength scale. The expression, constants, term set, and named diagnostics form its lowered ObjectiveSpec. Equation (2)

defines the entire minimized objective, not a residual around a hidden native loss.

For evaluator E_j , define $m_j(f; d, s) = E_j(\mathbf{x}_{d,s}(f))$ and let f_0 denote native DREAMPlace. CoEvoP&R orients every metric so that positive gain is better:

$$g_j(f; d, s) = o_j \frac{m_j(f; d, s) - m_j(f_0; d, s)}{v_j}, \quad o_j \in \{-1, +1\}, \quad (3)$$

where $o_j = -1$ for costs, $o_j = +1$ for WNS and TNS, and $v_j > 0$ is a fixed panel-specific scale. Density overflow measures placement feasibility; routing congestion and routed wirelength come only from the routed evaluator.

Search-panel evidence guides parent selection; final-panel metrics never enter prompts or archive fitness. Budgets B_A, B_B, B_C bound placement, timing, and routing calls. Selection of the returned objective is restricted to candidates with Tier-C evidence:

$$f^* = \arg \max_{f \in \mathcal{A}_G^C} S_{\text{final}}(f), \quad N_\ell \leq B_\ell, \quad \ell \in \{A, B, C\}, \quad (4)$$

where $\mathcal{A}_G^C$ is the routed subset of the final archive. Thus, search guidance and final evidence remain disjoint.

3.2 Archive-Conditioned Proposal

Evolution starts from a measured portfolio of native-equivalent, alternative wirelength-density, density-heavy, and routing-aware formulas. Valid symbolic seeds populate the islands of $\mathcal{A}_0$; native DREAMPlace remains an external, uneditable reference. Each island receives a measured eligible seed before LLM sampling.

The controller rotates across islands and samples a parent from the current island’s eligible MAP-Elites cells. Let $S_A(p)$ be the constrained Tier-A score defined below, $d_{\text{base}}(p) \in [0, 1]$ the distance from the nearest fixed mechanism, and $n_g(\text{sig}(p))$ the number of candidates with the same mechanism signature. The implemented novelty-aware distribution is

$$\begin{aligned} \pi_{\text{par}}(p \mid \mathcal{A}_g) &= \frac{w_g(p)}{\sum_{u \in \mathcal{P}_g} w_g(u)}, \\ w_g(p) &= \max\{\log(1 + [S_A(p)]_+), \epsilon\} \\ &\quad \times \frac{\frac{1}{2} + d_{\text{base}}(p)}{\sqrt{n_g(\text{sig}(p))}}. \end{aligned} \quad (5)$$

for p in parent pool $\mathcal{P}_g$, which contains admitted elites and bounded, behaviorally distinct near misses but excludes structural failures and severe regressions. The mechanism-count denominator prevents repeated sampling of one family.

The prompt never contains the full archive. Retrieval constructs

$$\begin{aligned} \mathcal{H}_g &= \{f_0, p_g\} \cup \text{Top}(\mathcal{A}_g) \cup \text{Diverse}(\mathcal{A}_g, \phi) \\ &\quad \cup \text{NearMiss}(\mathcal{A}_g) \cup \text{RelevantFail}(\mathcal{A}_g, p_g). \end{aligned} \quad (6)$$

The bounded sets provide a champion, high scorers, behaviorally diverse cells, useful tradeoffs, and failures relevant to the selected parent’s terms. Duplicate programs are removed unless measured outcomes differ. The controller then builds

$$\Pi_g = \left(\underbrace{r, \iota, b, \sigma}_{\text{fixed contract}}, \underbrace{p_g, u_g, \mathbf{m}_g, \mathcal{H}_g}_{\text{measured context}} \right), \quad (7)$$

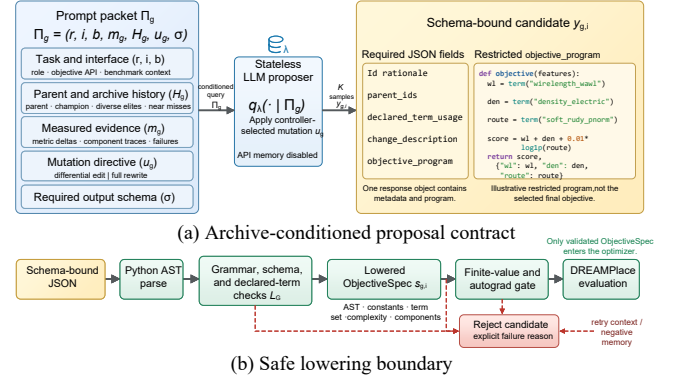


Figure 2: Proposal and lowering contracts. (a) A fixed objective interface and measured archive context condition K stateless LLM samples. (b) Schema, AST, term, and autograd checks lower a valid response into `ObjectiveSpec`; rejections return an explicit retry or negative-memory record.

where (r, ι, b, σ) fixes the task, objective API, target context, and JSON schema. Adaptive fields supply the parent, edit mode, evaluator scorecard, and retrieved programs. Gate thresholds remain hidden so that proposals use measured behavior rather than copy admission rules.

The controller normally requests a SEARCH/REPLACE edit and switches to a full rewrite after repeated static rejection. A stateless provider produces K independent responses from the same packet:

$$S_g = \{L_{\mathcal{G}}(y_{g,i}) \mid y_{g,i} \sim q_{\lambda}(\cdot \mid \Pi_g), L_{\mathcal{G}}(y_{g,i}) \neq \perp\}_{i=1}^K. \quad (8)$$

Each response contains id, rationale, parent_ids, declared_term_usage, change_description, and a complete objective_program. Provider retries repair malformed or rejected responses within the same sample. Deterministic coefficient siblings calibrate accepted routing mechanisms; they share one lineage, but each consumes evaluator budget.

Figure 2 exposes the proposal and lowering contracts. It also distinguishes independent samples from retries and generated code from the executable record.

3.3 Safe Objective Lowering

The lowering boundary makes the model’s action space finite. Static admission validates JSON, parses the Python AST as data, checks declared against observed terms, and lowers score and component expressions into `ObjectiveSpec`:

$$L_{\mathcal{G}}(y) = \begin{cases} s, & \text{static}_{\mathcal{G}}(y) = 1, \\ \perp, & \text{otherwise.} \end{cases} \quad (9)$$

The grammar permits constants, local assignments, named term calls, arithmetic, safe division, and `log1p`, `sqrt`, `square`, `softplus`, and `sigmoid`. It rejects control flow, imports, arbitrary calls, mutation, randomness, and direct feature access. AST depth is at most five, with 12 primitives per expression and six returned diagnostics.

Terms include weighted-average and log-sum-exp wirelength; electric and bell-shaped density; soft-RUDY mean and hotspot pressure; and long-net, pin-density, and pin-count-weighted pressure. Static compatibility requires exactly one wirelength and one density

family and rejects unavailable terms, baseline clones, and out-of-range routing coefficients. Nonlinear interactions remain expressible without duplicated anchors or ill-scaled objectives.

Dynamic admission occurs on the first DREAMPlace objective call. DREAMPlace evaluates only the lowered AST, differentiates the resulting scalar, and allows optimization to continue only when

$$\mathcal{F}_{\text{safe}} = \{f_s \mid \text{terms}(f_s) \subseteq \mathcal{T}, \quad |f_s(\mathbf{x}_0)| < \infty, \\ 0 < \|\nabla_{\mathbf{x}} f_s(\mathbf{x}_0)\|_2 < \infty, \quad \text{compat}(f_s) = 1\}. \quad (10)$$

Runtime guards repeat finite checks throughout placement and terminate at the first invalid state. The initial autograd gate establishes local executability; the guards catch later instability. Raw model code is never imported or executed.

3.4 Cost-Scaled Evaluation

Expensive evidence is allocated only after optimizer compatibility. Tier A runs each admitted objective on a multi-design DREAMPlace panel and records HPWL, density overflow, runtime, convergence, gradient behavior, per-design outcomes, and component trajectories. Tier B adds placement-RC timing when collateral exists: search uses FLUTE/Elmore, while final WNS/TNS use OpenTimer and are not called post-route timing. Tier C routes scheduled promotions through OpenROAD/ChiPBench and records global-route wirelength, routed congestion or overflow, available post-GRT timing, runtime, and failure stage.

Tier A first applies feasibility and robustness constraints. Let $\bar{g}_H, \bar{g}_D$ denote mean HPWL and density-overflow gains, $g_H^{\min}, g_D^{\min}$ their worst-design gains, and q_{HD} the fraction of designs improving both metrics. The admission indicator is

$$\chi_A(f) = \mathbb{I}[\bar{g}_H \geq -\delta_H, \quad \bar{g}_D \geq 0, \\ g_H^{\min} \geq -\delta_H^d, \quad g_D^{\min} \geq -\delta_D^d, \\ n_{\text{struct}} = n_{\text{severe}} = 0], \quad (11)$$

where fixed tolerances admit bounded HPWL and per-design numerical noise. The search score is

$$S_A(f) = \chi_A(f) \left(w_H[\bar{g}_H]_+ + w_D[\bar{g}_D]_+ + \frac{w_R}{1 + \bar{r}_A(f)} \right. \\ \left. + w_Q q_{HD}(f) - \eta c(f) \right), \quad (12)$$

where $[z]_+ = \max(z, 0)$, $\bar{r}_A$ averages rank across design-seed cells, and $c(f)$ is complexity. Weights and tolerances are fixed before a run. An elite must also beat the same-flow fixed-objective portfolio and Pareto-improve over its strongest HPWL/overflow member. Bounded distinct tradeoffs may enter $\mathcal{P}_g$ as near misses but cannot become routed finalists before passing the elite gate.

Cheap timing signals affect selection only when they add information beyond HPWL. For timing delta $q \in \{\Delta\text{WNS}, \Delta\text{TNS}\}$, the perturbation audit computes

$$\rho_H(q) = \max \{ |\text{Pearson}(\Delta\text{HPWL}, q)|, |\text{Spearman}(\Delta\text{HPWL}, q)| \}. \quad (13)$$

Let $a_C(q)$ indicate that held-out routed promotions preserve the proxy's rank direction for the corresponding downstream timing

metric. Its selection weight is

$$\gamma_q = a_C(q) \begin{cases} 0, & \rho_H(q) > 0.95, \\ \gamma_{\text{tie}}, & 0.70 \leq \rho_H(q) \leq 0.95, \\ \gamma_{\text{soft}}, & \rho_H(q) < 0.70, \end{cases} \quad 0 < \gamma_{\text{tie}} < \gamma_{\text{soft}} \leq 1. \quad (14)$$

Until enough routed samples exist, $a_C(q) = 0$ and timing remains diagnostic. Low HPWL correlation establishes non-redundancy; routed agreement establishes downstream relevance.

Every τ_C generations, at most k_C unrouted elites are promoted while $N_C < B_C$. The batch mixes high- S_A candidates with strong occupants of uncovered mechanism cells, then removes duplicate ASTs and signatures. Tier-C results are committed before Π_{g+1} . The final score compares only candidates with matched downstream evidence:

$$S_{\text{final}}(f) = \frac{1}{1 + \bar{r}_C(f)} - \eta_C c(f), \quad (15)$$

where $\bar{r}_C$ averages rank over reported post-GRT metrics and $\eta_C c(f)$ favors compact objectives only when ranks tie.

3.5 Archive and Closed-Loop Feedback

Each archive record contains parent, island, generation, prompt, provider snapshot and decoding settings, raw response, lowered AST, terms, components, complexity, tool configuration, per-design metrics, trajectories, artifacts, and failure reason. Structural failures and completed regressions remain queryable as negative memory but cannot occupy elite cells; near misses are exploration-only.

MAP-Elites assigns each parent-eligible candidate to

$$\phi(f) = (b_{\text{size}}(f), m(f), b_{\text{hpwl}}(f), b_{\text{dens}}(f), b_{\text{timing}}(f)), \quad (16)$$

where $m(f)$ identifies wirelength-density, route-pressure, long-net, pin-pressure, fanout-weighted, or hybrid mechanisms; missing timing maps to an unknown bucket. Each island retains the strongest eligible program per cell:

$$\mathcal{A}_{g+1}[b] = \arg \max_{u \in \mathcal{A}_g[b] \cup \{f\}} S_A(u), \quad b = \phi(f). \quad (17)$$

Ring migration periodically copies eligible cell elites into neighboring islands, where the same cell-replacement rule applies.

Feedback reconstruction converts tool records into compact, actionable evidence:

$$\mathbf{m}_{g+1} = \text{Summarize}(\Delta\text{rank}, \Delta\text{metric}, \mathbf{z}_{\text{design}}, \\ \mathbf{z}_{\text{comp}}, \mathbf{z}_{\text{fail}}, \mathbf{z}_{\text{route}}). \quad (18)$$

$\mathbf{z}_{\text{design}}$ reports per-design and worst-case behavior. $\mathbf{z}_{\text{comp}}$ stores each component's start/mid/end, range, mean, trend, and saturation status. $\mathbf{z}_{\text{fail}}$ records parser, gradient, convergence, and flow failures with repair lessons. $\mathbf{z}_{\text{route}}$ identifies which placement gains survive routing. Equation (6) selects the programs that accompany this scorecard in the next prompt.

CoEvoP&R couples objective evolution to progressively updated placement and routing evidence without training or mutating the EDA tools. Each routed promotion can change parent selection, retrieval, or failure context. Checkpointed prompts, responses, lowered records, metrics, and artifacts make lineages auditable despite nondeterministic providers. Algorithm 1 summarizes the loop.

Table 1: Main results and intra-family transfer on ChiPBench Nangate45. Values are mean improvements over native DREAMPlace across three seeds. Within each panel and column, **bold marks the best result and underline marks the second-best result; ties share the best mark.**

(a) HPWL reduction (%), $\uparrow$										(b) Density-overflow reduction (%), $\uparrow$									
Method	BP-FE	BP-BE	Swerv	Eth.	DFT	OR1200	VGA	Mor.	Mean	Method	BP-FE	BP-BE	Swerv	Eth.	DFT	OR1200	VGA	Mor.	Mean
DREAMPlace	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	DREAMPlace	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DP4.0-TD	-5.3	-1.7	-3.9	-0.4	-1.8	-4.6	0.3	-2.1	-2.4	DP4.0-TD	-2.4	0.7	-1.8	-2.3	-0.4	-1.1	0.4	-1.2	-1.0
Best expert	-2.4	3.7	-0.8	1.9	-1.2	2.6	4.1	-2.3	0.7	Best expert	4.7	6.8	0.9	3.4	1.6	4.2	2.7	3.7	3.5
BO-DSL	-0.7	3.8	1.2	4.6	-0.3	2.9	5.7	-0.4	2.1	BO-DSL	6.3	5.4	2.8	4.9	2.7	6.1	4.2	5.4	4.7
AutoDMP	<u>7.8</u>	8.6	11.3	12.7	5.4	<u>6.9</u>	14.8	9.7	9.7	AutoDMP	9.4	11.7	7.8	13.6	8.4	10.7	14.2	12.9	11.1
EvoPlace	2.4	6.8	5.1	9.7	0.9	4.3	14.6	6.1	6.2	EvoPlace	1.8	2.9	-0.6	2.7	0.3	2.4	3.6	0.7	1.7
LaMPlace	-19.4	-8.7	4.2	-11.6	3.8	-5.4	-26.3	-2.1	-8.2	LaMPlace	12.3	21.4	14.6	18.7	9.8	17.2	24.6	13.4	16.5
CoEvoP&R-E	8.4	<u>7.6</u>	38.4	17.8	9.3	7.2	31.7	14.6	16.9	CoEvoP&R-E	48.6	36.4	71.8	<u>15.4</u>	32.4	47.3	<u>18.9</u>	22.6	36.7
CoEvoP&R-L	6.2	5.7	<u>32.1</u>	<u>13.4</u>	<u>7.1</u>	5.4	<u>24.6</u>	<u>11.2</u>	<u>13.2</u>	CoEvoP&R-L	<u>42.8</u>	<u>31.7</u>	<u>62.4</u>	12.7	<u>27.6</u>	<u>40.8</u>	14.7	<u>18.4</u>	<u>31.4</u>
(c) WNS gain (ns), $\uparrow$										(d) TNS gain (ns), $\uparrow$									
Method	BP-FE	BP-BE	Swerv	Eth.	DFT	OR1200	VGA	Mor.	Mean	Method	BP-FE	BP-BE	Swerv	Eth.	DFT	OR1200	VGA	Mor.	Mean
DREAMPlace	0	0	0	0	0	0	0	0	0	DREAMPlace	0	0	0	0	0	0	0	0	0
DP4.0-TD	+0.51	<u>+0.56</u>	+0.20	+0.59	+0.11	+0.98	+0.19	+0.26	+0.43	DP4.0-TD	+208	+270	+95	+588	<u>+74</u>	+1785	+150	+220	+424
Best expert	-0.14	-0.21	-0.08	-0.18	-0.04	-0.28	-0.03	-0.09	-0.13	Best expert	-22.4	-34.6	-12.7	-68.4	-8.7	-178.6	-13.4	-27.2	-46
BO-DSL	-0.06	+0.04	-0.09	-0.03	+0.01	-0.13	+0.03	-0.01	-0.03	BO-DSL	-12.4	+6.8	-18.7	-14.6	+4.2	-52.7	+8.9	-6.4	-10.6
AutoDMP	+0.27	+0.28	+0.08	+0.22	+0.05	+0.59	+0.07	+0.14	+0.21	AutoDMP	+112	+114	+30	+324	+14	+840	+49	+80	+195
EvoPlace	+0.04	+0.06	-0.03	+0.03	+0.01	+0.07	-0.04	+0.02	+0.02	EvoPlace	+8.4	+14.7	-4.2	+22.6	+2.8	+38.4	-9.7	+6.3	+9.9
LaMPlace	+0.70	+0.78	+0.46	+0.72	+0.22	<u>+1.26</u>	+0.34	+0.42	<u>+0.61</u>	LaMPlace	+156	+138	+102	+384	+49	+1190	+84	+145	+281
CoEvoP&R-E	<u>+0.62</u>	+0.78	<u>+0.42</u>	<u>+0.62</u>	+0.52	+1.48	+0.87	<u>+0.28</u>	+0.70	CoEvoP&R-E	<u>+178</u>	+987	+648	<u>+495</u>	+108	<u>+1520</u>	+2846	+517	+912
CoEvoP&R-L	+0.46	+0.54	+0.32	+0.44	<u>+0.38</u>	<u>+0.58</u>	+0.21	+0.49		CoEvoP&R-L	+128	<u>+684</u>	<u>+467</u>	+348	+72	+1090	<u>+1968</u>	<u>+347</u>	<u>+638</u>

BP-FE and BP-BE denote black-parrot frontend and backend; DP4.0-TD denotes DREAMPlace 4.0-TD. E denotes target-design evolution; L denotes leave-one-design-out transfer. Each value uses DREAMPlace as the reference; higher is better. HPWL and density overflow use shared DREAMPlace reevaluation of each emitted mixed-size placement; WNS and TNS are OpenTimer placement-RC deltas. Baseline references: DREAMPlace and DREAMPlace 4.0-TD [13, 14]; BO-DSL uses TPE [4]; AutoDMP [2]; EvoPlace [27]; LaMPlace [9]. Best expert uses hand-coded HPWL-density, RUDY [24], and pin-density terms.

Algorithm 1 CoEvoP&R objective evolution

Require: $\mathcal{G}, \mathcal{D}_S, \mathcal{B}; G, K, B_A, B_B, B_C, \tau_C, k_C, \tau_{\text{mig}}$
Ensure: Archive $\mathcal{A}_G$ and routed objective f^*
1: $\mathcal{A}_0 \leftarrow \text{EvaluateSeeds}(\mathcal{B}, \mathcal{D}_S); N_A, N_B, N_C \leftarrow 0$
2: **for** $g = 0, \dots, G - 1$ **do**
3: $i \leftarrow g \bmod I; p_g \sim \pi_{\text{par}}(\cdot \mid \mathcal{A}_g^{(i)})$
4: $\mathcal{H}_g \leftarrow \text{Retrieve}(\mathcal{A}_g, p_g); u_g \leftarrow \text{DiffOrRewrite}(\mathcal{A}_g, p_g)$
5: $\Pi_g \leftarrow \text{Prompt}(p_g, u_g, \mathbf{m}_g, \mathcal{H}_g); \mathcal{Y}_g \leftarrow \{y_{g,j}\}_{j=1}^K \sim q_{\lambda}(\cdot \mid \Pi_g)$
6: **for** $y \in \mathcal{Y}_g$ **do**
7: $s \leftarrow L_{\mathcal{G}}(y)$
8: **if** $s = \perp$ **then**
9: RememberFailure(y); **continue**
10: **end if**
11: **for** $f \in \text{Calibrate}(s)$ **do**
12: **if** $N_A \geq B_A$ **then**
13: **break**
14: **end if**
15: $\mathbf{m}_A \leftarrow E_A(f, \mathcal{D}_S); N_A \leftarrow N_A + 1$
16: $\mathbf{m}_B \leftarrow \emptyset$; run $E_B(f)$ and increment N_B only if proxy-admitted and $N_B < B_B$
17: $\mathcal{A}_g \leftarrow \text{Insert}(\mathcal{A}_g, f, \mathbf{m}_A, \mathbf{m}_B)$
18: **end for**
19: **end for**
20: **if** $(g+1) \bmod \tau_C = 0$ and $N_C < B_C$ **then**
21: $C_g \leftarrow \text{Promote}(\mathcal{A}_g, k_C, B_C - N_C)$
22: $\mathbf{m}_C \leftarrow E_C(C_g); N_C \leftarrow N_C + |C_g|$
23: CommitRoutedFeedback($\mathcal{A}_g, \mathbf{m}_C$)
24: **end if**
25: $\mathcal{A}_{g+1} \leftarrow \text{Migrate}(\mathcal{A}_g)$ if scheduled, else $\mathcal{A}_g$
26: **end for**
27: **return** $\mathcal{A}_G, f^* = \arg \max_{f \in \mathcal{A}_G} S_{\text{final}}(f)$

4 Experiments and Results

4.1 Experimental Setup

The evaluation tests whether CoEvoP&R improves placement and post-GRT quality, transfers across benchmark families, depends on its safety/proxy/archive mechanisms, and remains robust across

LLM providers. The primary panel uses the eight ChiPBench Nangate45 designs in Table 1 [26]: bp_fe, bp_be, swerv_wrapper, ethernet, dft68, or1200, vga_lcd, and mor1kx. Generalization uses ICCAD 2015 Superblue designs 1, 3, 4, 5, 7, 10, 16, and 18 [12], plus ASAP7 gcd, ibex, and ariane for zero-shot transfer from Nangate45 to a 7-nm predictive process design kit [7].

Baselines span analytical placement (DREAMPlace and DREAMPlace 4.0-TD [13, 14]), expert density/HPWL/RUDY/pin-density objectives, same-grammar BO-DSL [4], and shared-pipeline reruns of AutoDMP [2], RoutePlacer [11], EvoPlace [27], and LaMPlace [9].

Each method-design cell in Tables 1 and 2 uses three seeds with DREAMPlace as zero. Table 1 reports DREAMPlace HPWL and density overflow plus OpenTimer placement-RC WNS/TNS. Table 2 reports post-GRT wirelength, overflow, and OpenTimer timing. A manifest keyed by design, method, seed, model, budgets, and metric records objective hashes, prompts, artifacts, route status, and failures, making every entry traceable. The default proposer is gpt-5.4; Section 4.5 tests model-family sensitivity.

4.2 Main Results on ChiPBench Nangate45

Table 1 combines target evolution and leave-one-design-out (LODO) transfer. Every entry is a three-seed, same-flow rerun; DREAMPlace provides HPWL/density overflow and OpenTimer provides placement-RC WNS/TNS. Means aggregate design-seed cells, and higher is better. Target-specific and LODO rows use distinct objectives. The Nangate45 champion reused unchanged for cross-family transfer in Section 4.3 is

$$f_{\text{NG45}}(\mathbf{x}) = [\text{Final formula}]. \quad (19)$$

Figure 3 compares a representative hard design under identical placement and GRT-overflow rendering. Target evolution improves mean HPWL/density overflow by 16.9%/36.7% versus best-baseline

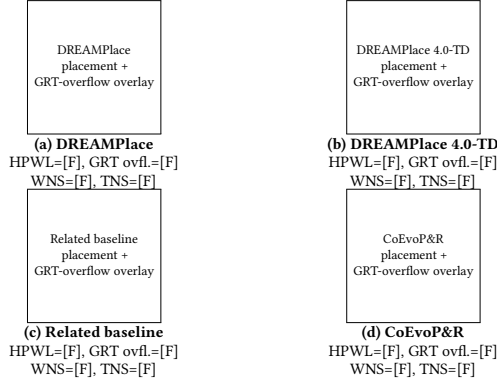


Figure 3: Qualitative placement comparison on a representative hard design. Each panel uses the same die outline, placement rendering, GRT-overflow overlay, and color scale. The visual audit checks whether CoEvoP&R’s metric gains correspond to reduced congestion artifacts while preserving layout structure.

means of 9.7%/16.5%, and gains 0.70/912 ns WNS/TNS. LODO retains 13.2%, 31.4%, 0.49 ns, and 638 ns.

Table 2: Cross-family generalization on ICCAD 2015 Superblue and ASAP7. All values are post-GRT improvements over DREAMPlace across three seeds. DP4.0-TD abbreviates DREAMPlace 4.0-TD; rWL denotes OpenROAD global-route wirelength. E denotes target-design evolution; T denotes zero-shot reuse of the Nangate45 champion. Within each metric and circuit, **bold marks the best result and underline marks the second-best result; ties share the same rank. Higher is better.**

Method	Metric	Superblue								ASAP7		
		sb1	sb3	sb4	sb7	sb10	sb16	sb18	gcd	ibex	ariane	
DREAMPlace	rWL (%)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	Cong. (%)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	WNS (Ans)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	TNS (Ans)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DP4.0-TD	rWL (%)	-0.4	-0.6	+0.2	-0.8	-0.3	+0.1	-0.5	+0.3	+0.4	-0.3	-0.8
	Cong. (%)	+0.2	-0.3	-0.4	+0.1	-0.2	-0.7	-0.1	+0.4	+0.6	+0.2	-0.4
	WNS (Ans)	+4.4	+16.8	+8.7	+21.7	+5.2	+1.9	+4.7	+8.3	+0.06	+0.18	+0.34
	TNS (Ans)	+16700	+3370	+5210	+11300	+9810	+7040	+31400	+4320	+32	+187	+584
RoutePlacer	rWL (%)	+0.1	-0.3	+0.1	+0.3	-0.1	+0.9	-0.1	+0.2	+0.4	-0.6	+1.8
	Cong. (%)	+3.1	+1.7	+4.6	+3.9	+3.9	+43.2	+1.4	+3.7	+6.4	+18.7	+24.6
	WNS (Ans)	-0.6	-0.4	+0.2	-0.4	-0.8	-1.2	-0.3	-0.7	-0.02	-0.14	-0.34
	TNS (Ans)	-1400	-800	+200	-1200	-1600	-3400	-800	-1400	-12	-87	-294
EvoPlace	rWL (%)	-2.4	-3.6	+0.4	-2.8	+1.4	-3.2	-1.8	+0.6	+1.2	-8.4	-13.4
	Cong. (%)	-3.4	-8.6	-2.4	-4.2	+0.4	-12.4	-6.8	-1.2	+0.4	-14.2	-18.4
	WNS (Ans)	+0.2	+0.4	+0.4	-0.4	+0.6	-0.8	-0.4	+0.4	+0.02	-0.24	-0.48
	TNS (Ans)	-600	-1400	+400	-1200	+300	-2200	-1400	+200	+4	-124	-487
LaMPlace	rWL (%)	-4.6	-6.8	+14.2	-8.4	+0.4	-2.4	+11.8	+22.4	-6.8	-18.4	-22.4
	Cong. (%)	+18.4	+14.6	+42.4	+15.8	+19.4	+20.6	+48.6	+32.4	+2.4	-8.7	-14.2
	WNS (Ans)	+2.4	+6.8	+3.4	+8.4	+3.6	+1.2	+2.8	+0.4	+0.04	-0.16	-0.32
	TNS (Ans)	+2400	+6800	+2400	+8400	+3800	+1400	+1800	+1200	-18	-94	-246
CoEvoP&R-E	rWL (%)	+5.4	+6.2	+4.8	+7.2	+5.4	+3.6	+6.4	+4.2	+5.4	+8.2	+10.6
	Cong. (%)	+26.4	+19.4	+22.6	+22.4	+21.7	+34.7	+9.8	+28.4	+14.6	+22.4	+26.8
	WNS (Ans)	+5.8	+14.8	+9.4	+12.8	+7.6	+2.6	+6.2	+6.4	+0.24	+0.42	+0.68
	TNS (Ans)	+18800	+38400	+18700	+67200	+36400	+8400	+14600	+6800	+148	+468	+824
CoEvoP&R-T	rWL (%)	+1.4	+3.8	+2.6	+4.2	+3.2	+1.8	+3.6	+2.4	+2.4	+3.8	+5.2
	Cong. (%)	+12.4	+18.8	+14.6	+18.7	+15.4	+22.8	+7.8	+19.4	+6.4	+11.8	+14.6
	WNS (Ans)	+1.4	+8.2	+5.4	+7.4	+4.6	+1.8	+4.6	+4.2	+0.08	+0.22	+0.34
	TNS (Ans)	+2400	+18400	+9600	+32400	+14200	+4800	+7600	+4600	+58	+204	+368

All rows use the shared OpenROAD post-GRT flow. rWL is global-route wirelength, Cong. is total global-routing overflow reduction, and WNS/TNS come from OpenTimer on the post-GRT routed netlist. Baseline references follow Table 1: DREAMPlace and DREAMPlace 4.0-TD [13, 14]; RoutePlacer [11]; EvoPlace [27]; LaMPlace [9]. All tabulated values are shared-pipeline reruns. LaMPlace publishes Innovus post-detail-route results for superblue4, superblue16, and superblue18; those values provide a directional, not numerically equivalent, comparison to our post-GRT flow.

4.3 Generalization

Table 1 measures intra-family LODO; Table 2 compares target-design evolution (E) with zero-shot reuse of the exact Nangate45 objective (T) on Superblue and ASAP7. Target metrics are excluded from transfer prompts and parent selection.

On Superblue, E averages 5.4% rWL, 23.2% GRT-overflow reduction, 8.2 ns WNS, and 26,163 ns TNS gain. T retains 2.9%, 15.7%, 4.7 ns, and 11,750 ns, respectively, or 45%–68% of E. LaMPlace leads E in rWL and congestion on superblue4/16/18, while DP4.0-TD leads TNS on superblue16. On ASAP7, LaMPlace regresses while both CoEvoP&R variants remain positive, supporting symbolic cross-node transfer. Superblue1 is the clearest transfer-limit case.

4.4 Proxy Audit

Table 3: Timing-proxy admission audit on the four-design stress panel. Perturbations use 100, 250, 500, 1000, and 2000 DBU. Thresholds follow Section 3.4; Influence reports the policy after the routed agreement gate.

Metric pair	Med. Pearson	Med. Spearman	A/T/R	Influence
Δ WNS vs. Δ HPWL	0.58	0.54	3/1/0	soft gate
Δ TNS vs. Δ HPWL	0.82	0.78	1/2/1	tie-breaker

Table 3 reports Pearson/Spearman correlation and admitted/tie-weighted/rejected counts (A/T/R) on bp_fe, swerv, ethernet, and ariane133. These correlations test HPWL non-redundancy; held-out agreement with subsequent Tier-C timing determines whether the indicated weight is activated. The ungated timing-proxy control tests this two-part policy.

4.5 Causal Controls and Model Sensitivity

Table 4: Claim-aligned controls and model robustness on the four-design stress panel. Proposal and archive controls match candidate and Tier-C budgets; direct Tier C routes every valid candidate. Valid/Acc.: valid/accepted objectives; H/D: HPWL/density-overflow reduction; W/T: WNS/TNS gain. Score follows Equation (12); Cost is total GPU-hours.

Configuration	Valid/Acc.	Score	H/D (%)	W/T (Ans)	Tier-C evals	Cost (GPU-h)
Full CoEvoP&R						
68/12	4.87	18.4/42.6	+0.68/+624	24	84	
<i>Proposal: objective-search policy</i>						
Random DSL evolution	42/4	1.24	2.4/4.8	+0.08/+42	24	68
One-shot LLM sampling	58/8	2.14	6.4/12.4	+0.24/+184	24	42
Score-only LLM evolution	62/10	3.42	12.6/24.8	+0.42/+318	24	76
<i>Evaluation: interface and evidence policy</i>						
Prompt-only free-form interface	24/4	1.86	5.4/11.4	+0.16/+112	24	72
Tier-A-only selection	68/18	2.68	8.4/14.6	+0.24/+186	0	12
Direct Tier C (all valid)	68/8	3.14	10.6/22.4	+0.34/+348	68	168
Ungated timing proxy	68/14	3.86	14.8/32.6	+0.48/+412	24	86
<i>Archive: retained search state</i>						
Champion-only scalar history	62/10	2.42	8.6/16.4	+0.28/+218	24	82
Top-k scalar archive	64/11	3.14	12.4/24.8	+0.38/+312	24	84
MAP-Elites, metrics only	66/12	3.68	15.2/32.4	+0.48/+412	24	84
<i>Model-family sensitivity</i>						
gpt-5.4 (default)	68/12	4.87	18.4/42.6	+0.68/+624	24	84
gpt-4.1-mini	62/10	3.87	14.8/34.2	+0.52/+468	24	68
claude-opus-4-8	66/12	4.62	17.4/40.8	+0.64/+584	24	92
qwen3.7-max	58/9	3.24	12.6/26.8	+0.44/+368	24	74

Table 4 asks one causal question per stage of Figure 1. Proposal controls separate the LLM prior, iterative search, and execution feedback through random DSL evolution, equal-sample one-shot generation, and score-only evolution. Evaluation controls compare the machine-checkable interface against prompt-only free-form proposals, Tier-A-only selection, routing every valid candidate at Tier C, and an ungated timing proxy. Every row sends only lowered, autograd-valid objectives to DREAMPlace; raw proposals remain data.

Archive controls compare champion-only scalars, top- k scalar history, and a metrics-only MAP-Elites archive against full feedback reconstruction. Proposal and archive rows match candidate and Tier-C budgets. Model rows hold prompts, mutation schedules, and evaluator budgets fixed while comparing the default proposer, an efficient same-provider model, and frontier models from Anthropic and Qwen. These controls attribute performance to proposal search, evidence allocation, and retained execution state.

5 Conclusion

CoEvoP&R turns routing-aware placement-objective design into a search over complete executable replacement objectives. The framework combines a safe objective interface, cost-scaled placement and routing evaluation, proxy admission, and a routed-feedback archive loop. These components let LLM-generated objectives enter analytical placement without arbitrary code execution or unchecked proxy feedback.

Across eight ChipBench Nangate45 designs, target-evolved objectives reduce HPWL by 16.9% and density overflow by 36.7% over native DREAMPlace while improving OpenTimer placement-RC WNS by 0.70 ns and TNS by 912 ns. LODO transfer retains 13.2% HPWL reduction, 31.4% density-overflow reduction, 0.49 ns WNS gain, and 638 ns TNS gain. Superblue and ASAP7 results further measure target-design evolution and zero-shot cross-family transfer under a shared post-GRT flow.

CoEvoP&R gives EDA objective design a measured search interface. Many physical-design loops rely on cheap proxy signals while optimizing for expensive downstream costs. CoEvoP&R evolves symbolic objectives under that hierarchy while keeping optimizer execution fail-closed and proxy influence auditable.

References

- [1] Anthony Agnesina, Rongjian Liang, Geraldo Pradipta, Anand Rajaram, and Haoxing Ren. 2025. GOALPlace: Begin with the End in Mind. In *Proceedings of the International Symposium on Physical Design*. ACM, New York, NY, USA, 2–10. doi:10.1145/3698364.3705342
- [2] Anthony Agnesina, Puranjay Rajvanshi, Tian Yang, Geraldo Pradipta, Austin Jiao, Ben Keller, Bruce Khailany, and Haoxing Ren. 2023. AutoDMP: Automated DREAMPlace-based Macro Placement. In *Proceedings of the International Symposium on Physical Design*. ACM, New York, NY, USA, 149–157. doi:10.1145/3569052.3578923
- [3] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jiajia Lee, Uday Mallappa, Marina Neeseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Le Wang, Zhiru Wang, Mingyu Woo, and Bangqi Xu. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proceedings of the 56th Annual Design Automation Conference*. ACM, New York, NY, USA, 76:1–76:4. doi:10.1145/3316781.3326334
- [4] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., Granada, Spain, 2546–2554.
- [5] Magi Chen and Ting-Chi Wang. 2025. HyperPlace: Harnessing a Large Language Model for Efficient Hyperparameter Optimization in GPU-Accelerated VLSI Placement. *ACM Transactions on Design Automation of Electronic Systems* 30, 6, Article 98 (2025), 27 pages. doi:10.1145/3733601
- [6] Chung-Kuan Cheng, Andrew B. Kahng, Ilgeun Kang, and Lutong Wang. 2019. RePlace: Advancing Solution Quality and Routability Validation in Global Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 9 (2019), 1717–1730. doi:10.1109/TCAD.2018.2859220
- [7] Lawrence T. Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandrasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm FinFET Predictive Process Design Kit. *Microelectronics Journal* 53 (2016), 105–115. doi:10.1016/j.mejo.2016.04.006
- [8] Wenji Fang, Yao Lu, Shang Liu, Qijun Zhang, Ceyu Xu, Lisa Wu Wills, Hongce Zhang, and Zhiyao Xie. 2023. MasterRTL: A Pre-Synthesis PPA Estimation Framework for Any RTL Design. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. IEEE, San Francisco, CA, USA, 1–9. doi:10.1109/ICCAD57390.2023.10323951 arXiv:2311.08441.
- [9] Zijie Geng, Jie Wang, Ziyang Liu, Siyuan Xu, Zhenhao Tang, Shixiong Kai, Mingxuan Yuan, Jianye Hao, and Feng Wu. 2025. LaMPlace: Learning to Optimize Cross-Stage Metrics in Macro Placement. In *International Conference on Learning Representations*. OpenReview.net, Singapore.
- [10] Raghu Vamsi Hemadri, Jitendra Bhandari, Andre Nakkab, Johann Knechtel, Badri P. Gopalan, Ramesh Narayanaswamy, Ramesh Karri, and Siddharth Garg. 2025. VeriLoC: Line-of-Code Level Prediction of Hardware Design Quality from Verilog Code. In *Advances in Neural Information Processing Systems*. Neural Information Processing Systems Foundation, San Diego, CA, USA and Mexico City, Mexico. doi:10.48550/arXiv.2506.07239 arXiv:2506.07239.
- [11] Yunbo Hou, Haoran Ye, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2024. RoutePlacer: An End-to-End Routability-Aware Placer with Graph Neural Network. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 1085–1095. doi:10.1145/3637528.3671895
- [12] Myung-Chul Kim, Jin Hu, Jiajia Li, and Natarajan Viswanathan. 2015. ICCAD-2015 CAD Contest in Incremental Timing-Driven Placement and Benchmark Suite. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. IEEE, Austin, TX, USA, 921–926. doi:10.1109/ICCAD.2015.7372671
- [13] Peiyu Liao, Siting Liu, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. DREAMPlace 4.0: Timing-Driven Global Placement with Momentum-Based Net Weighting. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition*. IEEE, Antwerp, Belgium, 939–944. doi:10.23919/DAT54114.2022.9774725
- [14] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Bruce Khailany, and David Z. Pan. 2019. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. In *Proceedings of the 56th Annual Design Automation Conference*. ACM, New York, NY, USA, 1–6. doi:10.1145/3316781.3317803
- [15] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Vienna, Austria, 32201–32223.
- [16] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Cardei, Kishor Chen, Rene Correa, et al. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. arXiv preprint arXiv:2311.00176.
- [17] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Jing Wang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCode: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. doi:10.1109/TCAD.2024.3483089
- [18] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. In *International Conference on Learning Representations*. OpenReview.net, Vienna, Austria.
- [19] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A Graph Placement Methodology for Fast Chip Design. *Nature* 594 (2021), 207–212. doi:10.1038/s41586-021-03544-w
- [20] Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Boris Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv preprint arXiv:2506.13131.
- [21] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625 (2024), 468–475. doi:10.1038/s41586-023-06924-6

- [22] Asankhaya Sharma. 2025. OpenEvolve: An Open-Source Evolutionary Coding Agent. <https://github.com/algorithmicsuperintelligence/open evolve>.
- [23] Parshin Shojaei, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K. Reddy. 2025. LLM-SR: Scientific Equation Discovery via Programming with Large Language Models. In *International Conference on Learning Representations*. OpenReview.net, Singapore.
- [24] Peter Spindler and Frank M. Johannes. 2007. Fast and Accurate Routing Demand Estimation for Efficient Routability-Driven Placement. In *Proceedings of the Conference on Design, Automation and Test in Europe*. IEEE, Nice, France, 1226–1231. doi:10.1109/DATE.2007.364463
- [25] Ikechukwu Uchendu, Swati Goel, Karly Hou, Ebrahim Songhori, Kuang-Huei Lee, Joe Wenjie Jiang, Vijay Janapa Reddi, and Vincent Zhuang. 2026. See It to Place It: Evolving Macro Placements with Vision-Language Models. arXiv preprint arXiv:2603.28733.
- [26] Zhihai Wang, Zijie Geng, Zhaojie Tu, Jie Wang, Yuxi Qian, Zhixuan Xu, Ziyang Liu, Siyuan Xu, Zhentao Tang, Shixiong Kai, Mingxuan Yuan, Jianye Hao, Bin Li, Yongdong Zhang, and Feng Wu. 2025. Benchmarking End-To-End Performance of AI-Based Chip Placement Algorithms. In *Advances in Neural Information Processing Systems*. Neural Information Processing Systems Foundation, San Diego, CA, USA and Mexico City, Mexico. Datasets and Benchmarks Track, arXiv:2407.15026.
- [27] Xufeng Yao, Jiaxi Jiang, Yuxuan Zhao, Peiyu Liao, Yibo Lin, and Bei Yu. 2026. EvoPlace: Evolution of Optimization Algorithms for Global Placement via Large Language Models. Accepted to IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems; arXiv:2504.17801. doi:10.1109/TCAD.2026.3652945